

Automation Developer Experience

Ansible Development Workspaces

Name

Title

Agenda

- ▶ Common goals and challenges
- ▶ Ansible content types
- ▶ Ansible development tooling
- ▶ Ansible development workspaces
- ▶ Dev Spaces for power users
- ▶ Ansible testing

Common Goals and Challenges

Common Automation Drivers and Goals



Drivers

- ▶ Dependency on tribal knowledge and poorly documented processes
- ▶ Legacy systems that require special care and feeding
- ▶ Dynamic workforce with automation needs across multiple teams
- ▶ Many products and technologies that need to work together

Goals



- ▶ Greater speed and agility in rolling out services and features
- ▶ Fewer service disruptions
- ▶ Less time spent on low value activities and unproductive work
- ▶ Freedom to switch technologies and infrastructure with business needs

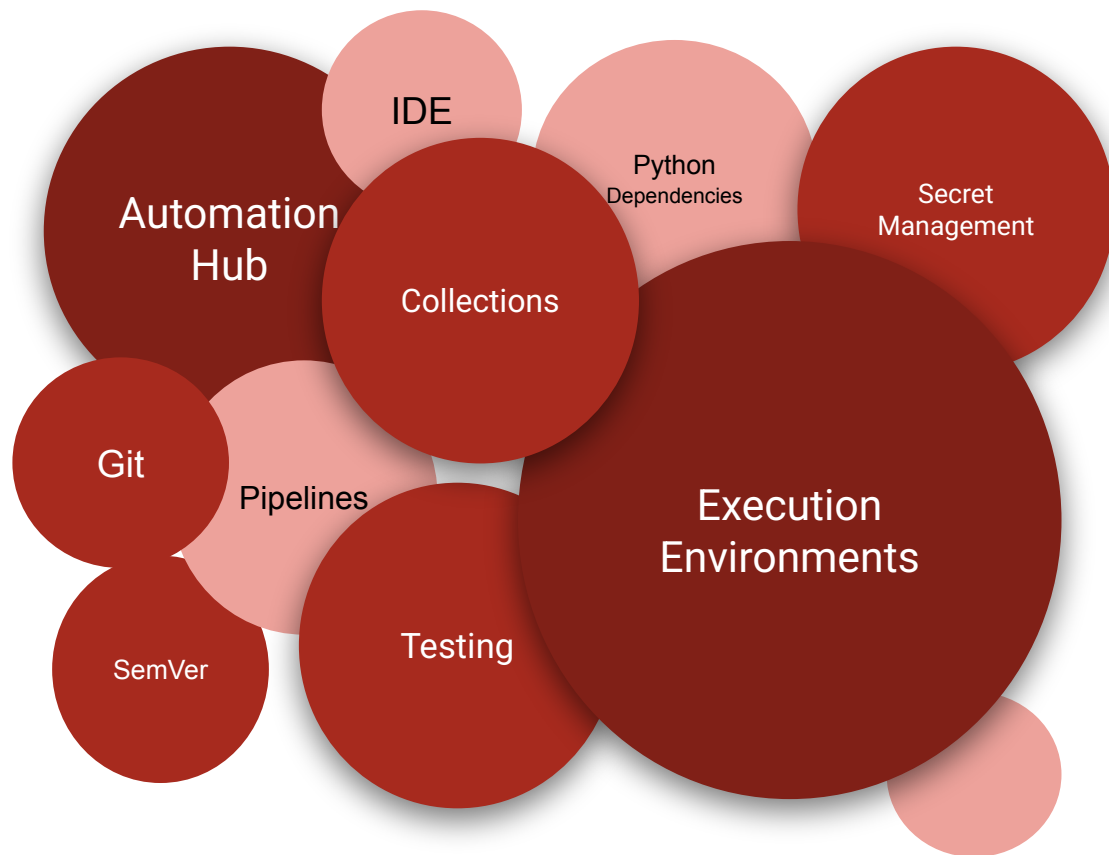
Ansible is Simple and Easy

```
---
- name: Install and start Apache
  hosts: webserver-01
  tasks:
    - name: Install httpd package
      package:
        name: httpd
        state: present

    - name: Start and enable httpd service
      service:
        name: httpd
        state: started
        enabled: true
```

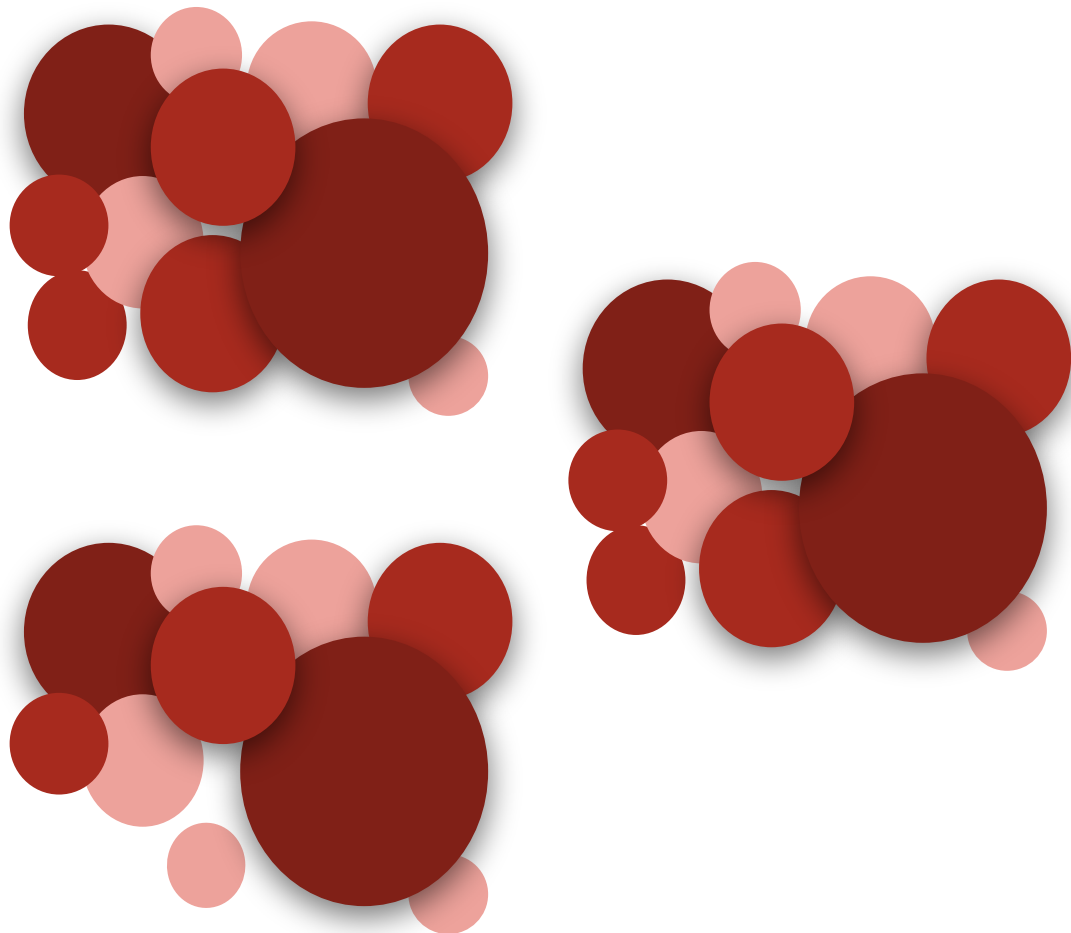
- **Easy**
 - Causing or involving little difficulty or discomfort
- **Simple**
 - Free of secondary complications

Scaling Automation Development to the Enterprise



- Not as easy
 - You're a developer now!
 - New concepts and tools to learn
- Not as simple
 - Multiple technologies
 - Multiple dependencies

Uncoordinated Automation Development Tooling



- Lower return on investment
- Reduced adoption
- Less reliable infrastructure
- Inconsistent code quality
- Poor user experience
- Inferior / inconsistent documentation

Common Challenges with Automation Adoption

Lack of standards and common tooling



I'm not comfortable using git or any of these other tools and really don't know where to start.

Trouble Getting Started



What do I need to install on my workstation and who can give me access to do it?

Missing Dependencies



How can I have confidence that the automation code is safe?

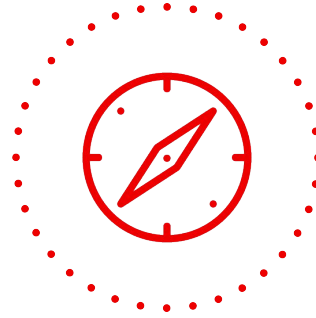
Lack of Trust

Common Challenges



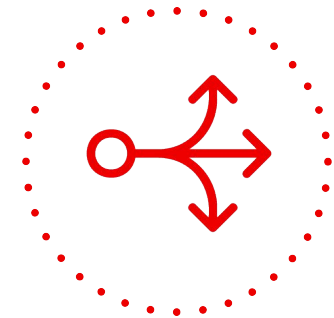
Automation Developers

- May not identify as a developer
- Minimal experience with software development tooling and processes
- Often less familiar with software testing



Local Environment

- Integrated Development Environment needed
- Ansible Development Tools require Linux
- Containers / VMs needed
- Local security enforcement



Environmental Differences

- “Works on my machine”
- Shared secrets
- Version mismatches
- Layered and forgotten configurations not captured as code

Onboarding Example

Services engagement at a large financial institution

Project Kick-off

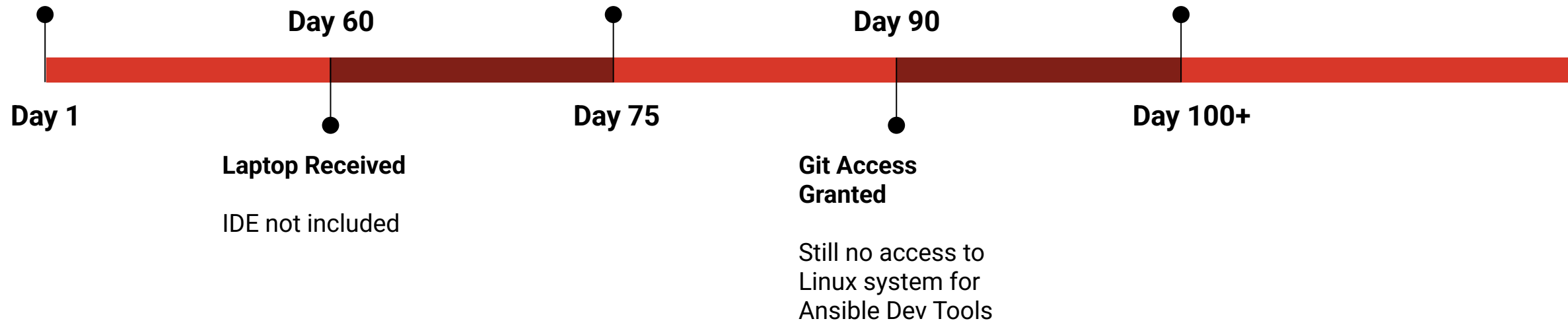
Tickets submitted
for hardware

Process Abandoned

Third party (external)
systems needed to meet
project timelines

VSCode Installed

Software request approved



Patterns for Success

Make it easy:



- Prioritize tools users are already familiar with
- Provide working examples as a starting point
- Facilitate communities and a culture of sharing
- Lego brick components

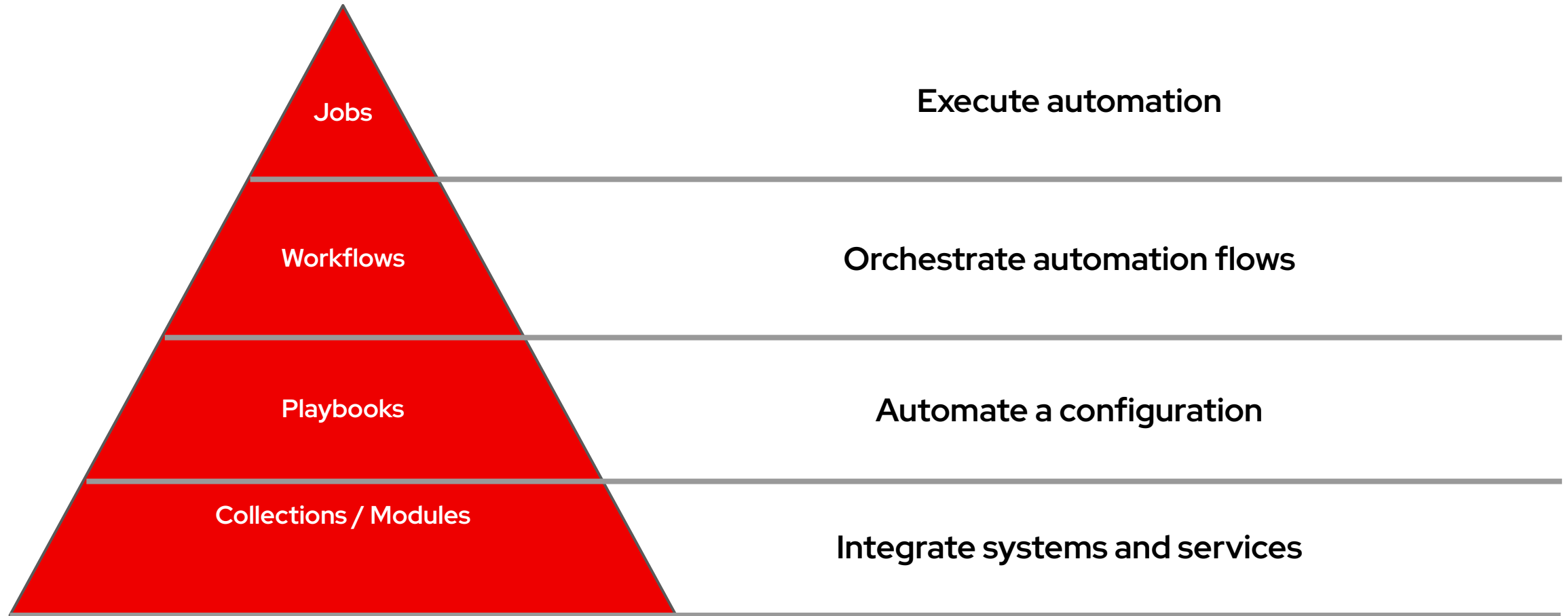
Patterns for Success

Make it Simple:

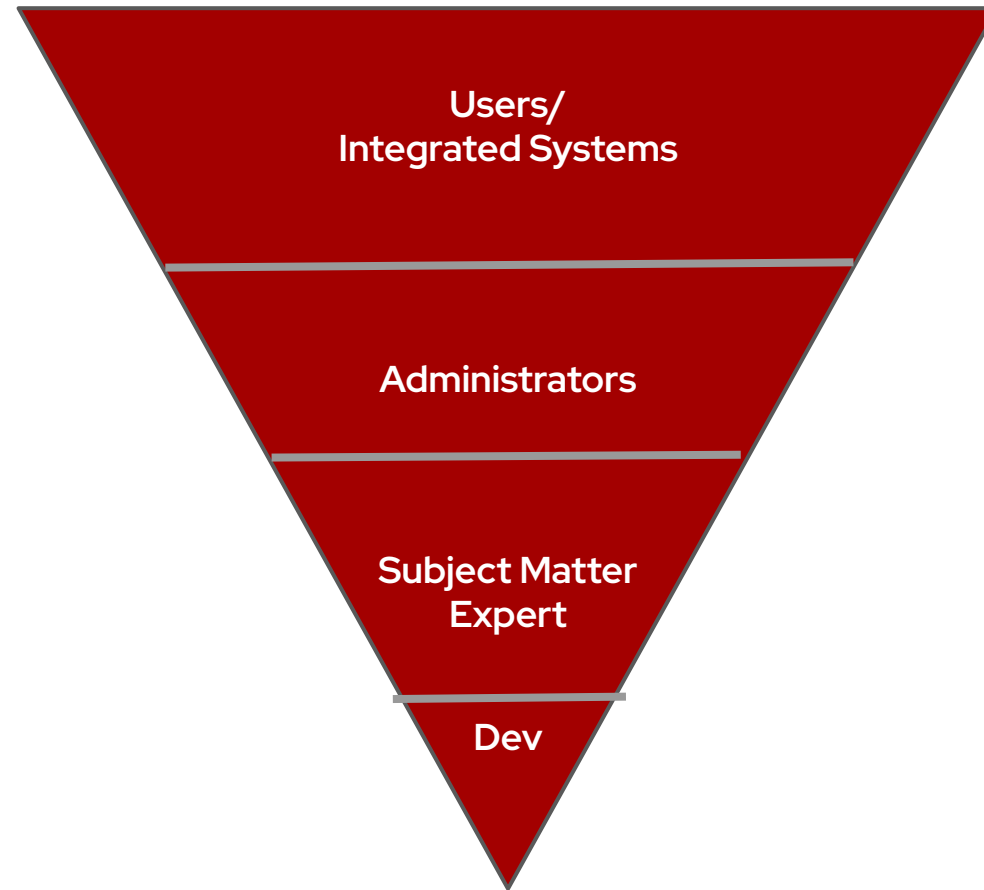
- Limit the things a user needs to understand to get started
- Centralize and standardize dependencies
- Be consistent

Ansible Content Types

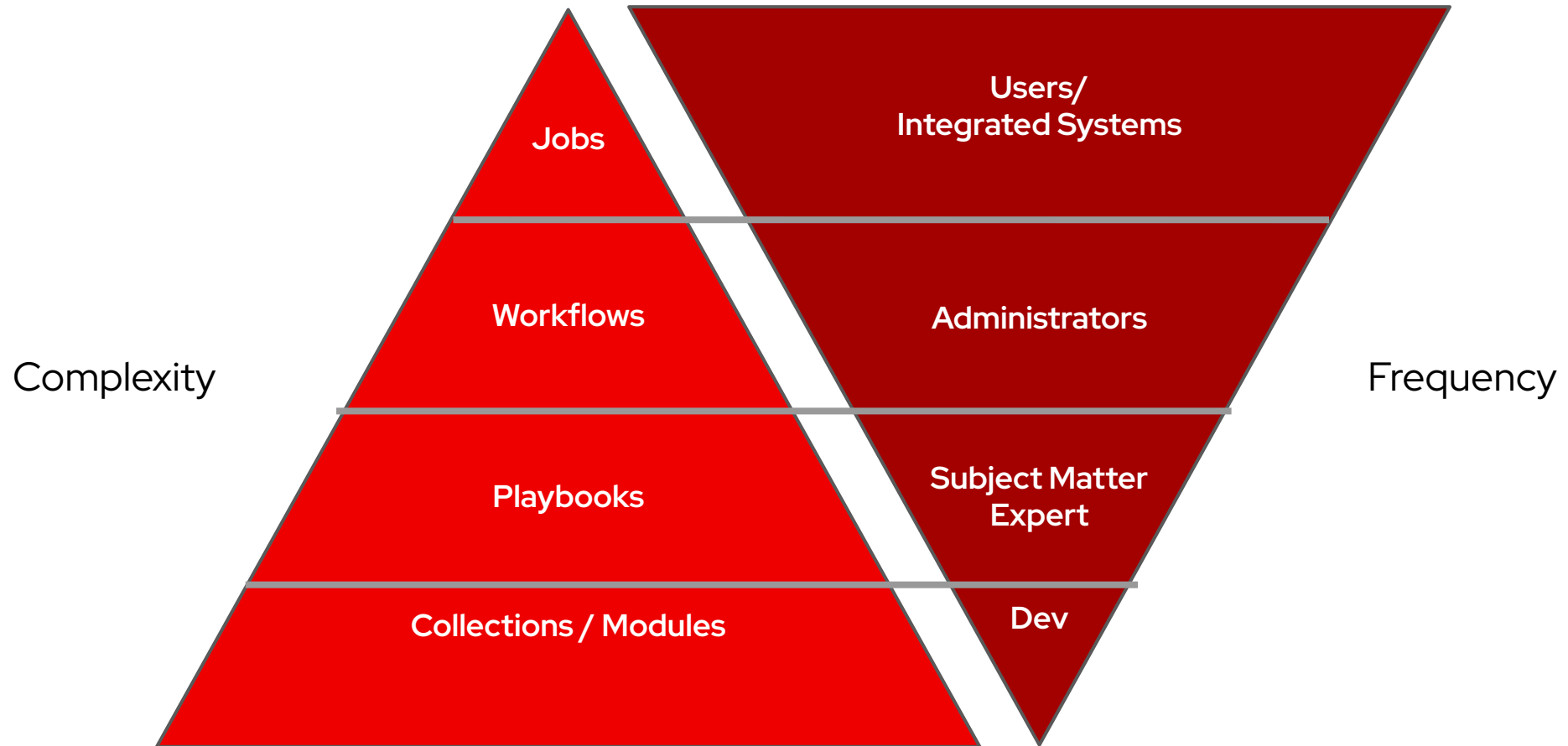
Ansible Content Types



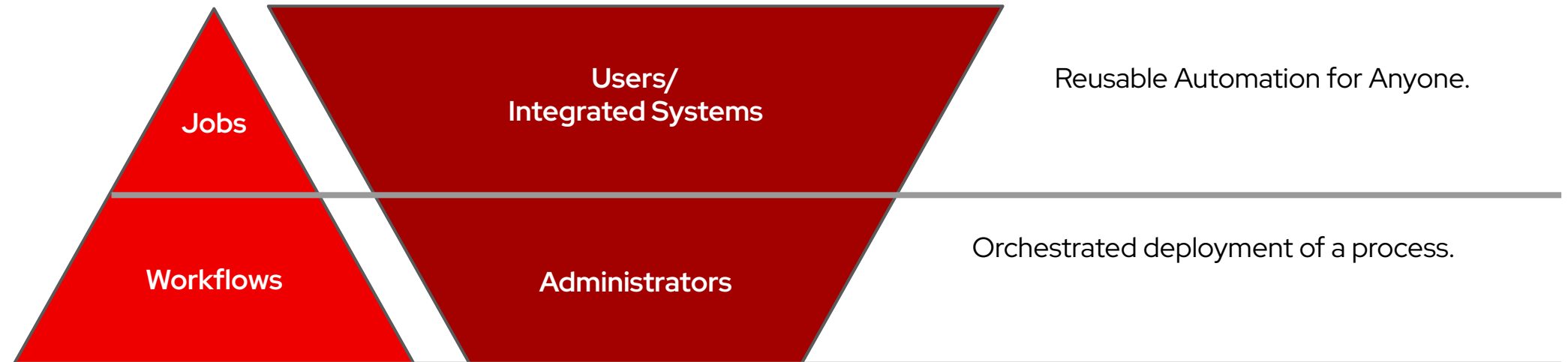
Ansible Content Personas



Ansible Content Personas



Ansible for the Enterprise | Execution

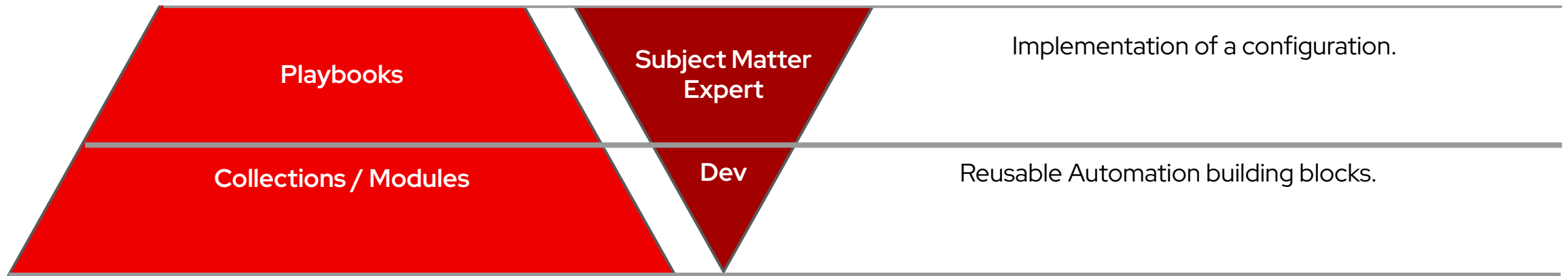


Red Hat
Ansible Automation
Platform

Ansible for the Enterprise | **Development**

What is the automation development process?

Do developers have the tools necessary to create quality code?



Ansible for the Enterprise | **Development**

Windows Only Development Challenges



-  Ansible
-  Ansible-lint
-  Ansible-navigator
-  Ansible-builder
-  Molecule
-  Ansible-test
-  Syntax highlighting
-  And more

Ansible for the Enterprise | Development

Windows Only Development Workarounds

		Centrally Managed	Self Service	Drawbacks
1	Remote SSH	×	×	• Requires an additional Linux host • Capacity issues on shared hosts (mostly disk) • Authentication issues (ie puttyCAC) • Added complexity & single point of failure
2	WSL Version 1	×	×	• Requires admin rights to install • Very slow • Rarely approved by security • Usually unmanaged and unpatched
3	WSL Version 2	×	×	• Requires admin rights to install • Network issues & local agent issues • Rarely approved by security • Usually unmanaged and unpatched
4	Docker Desktop	—	×	• Requires admin rights to install • Additional Licensing • Rarely approved by security • Usually unmanaged and unpatched
5	Podman Desktop	—	×	• Requires admin rights to install • Requires WSL • May not be on trusted software lists • Usually unmanaged and unpatched

Ansible Development Tooling

Ansible development tools (ansible-dev-tools)

Supported, enterprise-grade components to enable creating, testing, and deployment of Ansible

Create

ansible-creator

Scaffold projects and content

ansible-navigator

Develop and troubleshoot with execution environments

Ansible development environment (ade)

Build and manage virtual environments for content development

Test

Ansible Molecule

Functional test runner for collections, playbooks and roles

ansible-lint

Identify and correct stylistic errors and anti-patterns

pytest-ansible

Added functions for module and plugin Python code unit testing

tox-ansible

Test collections under different Python interpreters and Ansible core versions

Deploy

ansible-builder

Build Ansible execution environments

ansible-sign

Sign and verify Ansible content



Ansible VS Code extension and Dev Container

Normalize and Isolate Development Environments

- ▶ Create containerized development environments with all Ansible development tools included
- ▶ Compatibility with macOS, Windows and Linux
- ▶ Access Microsoft Dev Containers extension through VS Code marketplace

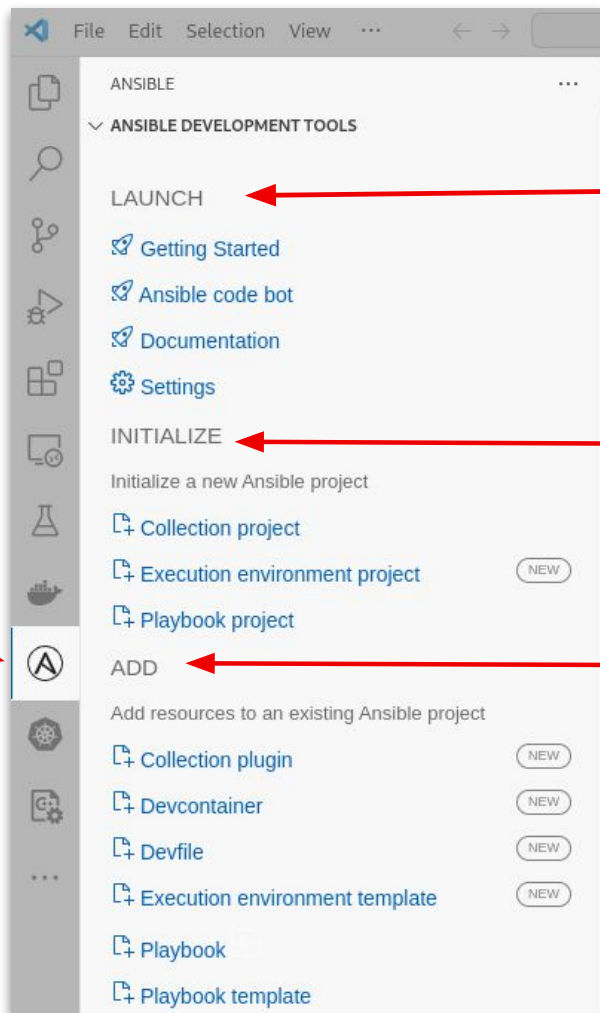


Active dev container
in VS Code



Ansible VS Code extension

Integrating tools into a unified UI



Learn more about the extension and Ansible development tools

Initialize projects with ansible-creator

Generate playbooks, add project resources, and build EEs

Create new Ansible collection

Streamlining automation

Namespace *

Collection *

Collection name: acme.acme_tools

Init path

Project path: /root/.ansible/collections/ansible_collections/acme/acme_tools

Forms for creating content via the VS Code UI

Ansible Development Workspaces

Scaling Automation for the Enterprise | **Development**



Red Hat
Advanced
Developer Suite

Advanced Development Capabilities

Internal Development Portal | Secure Software Delivery | Developer Tools

Playbooks

Subject Matter
Expert

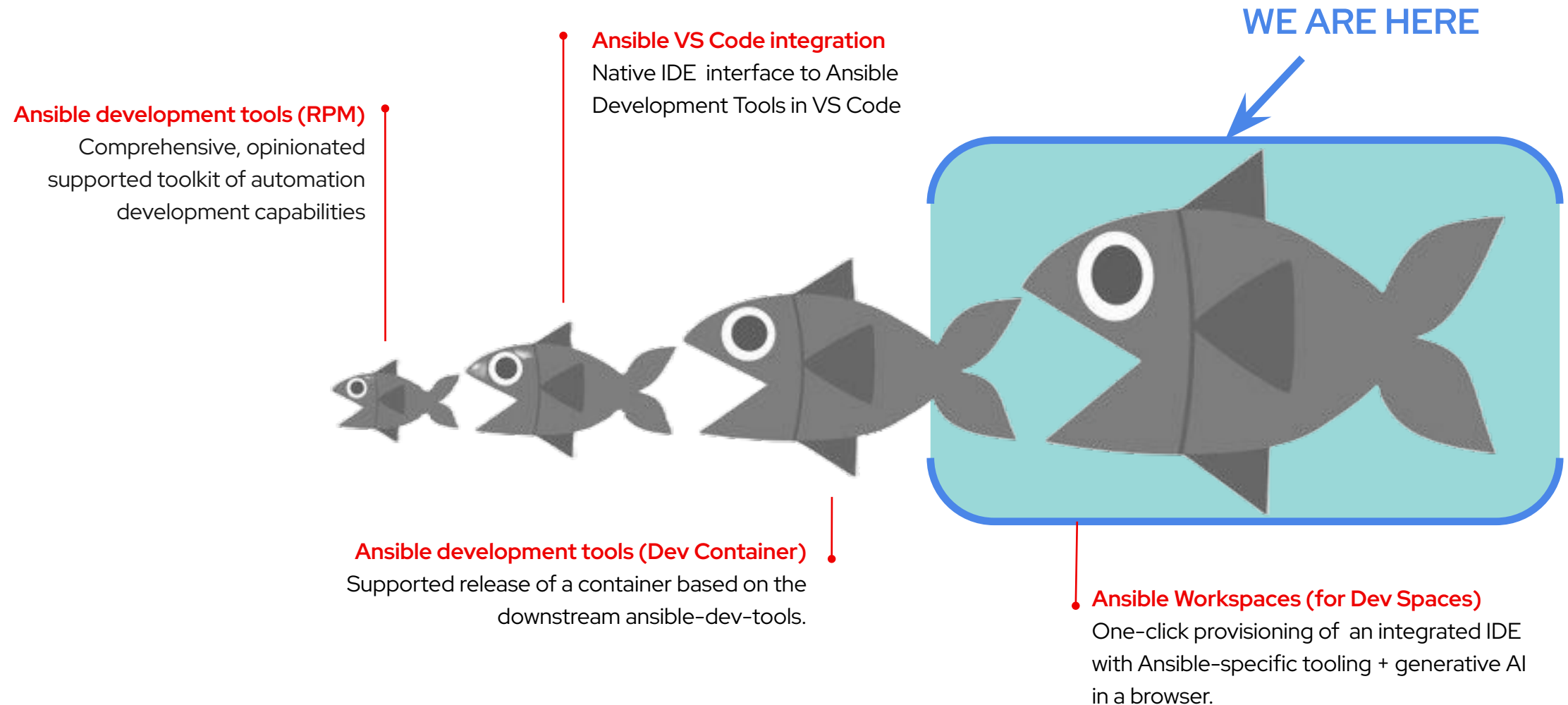
Implementation of a configuration.

Collections / Modules

Dev

Reusable Automation building blocks.

The Ansible development tools experience

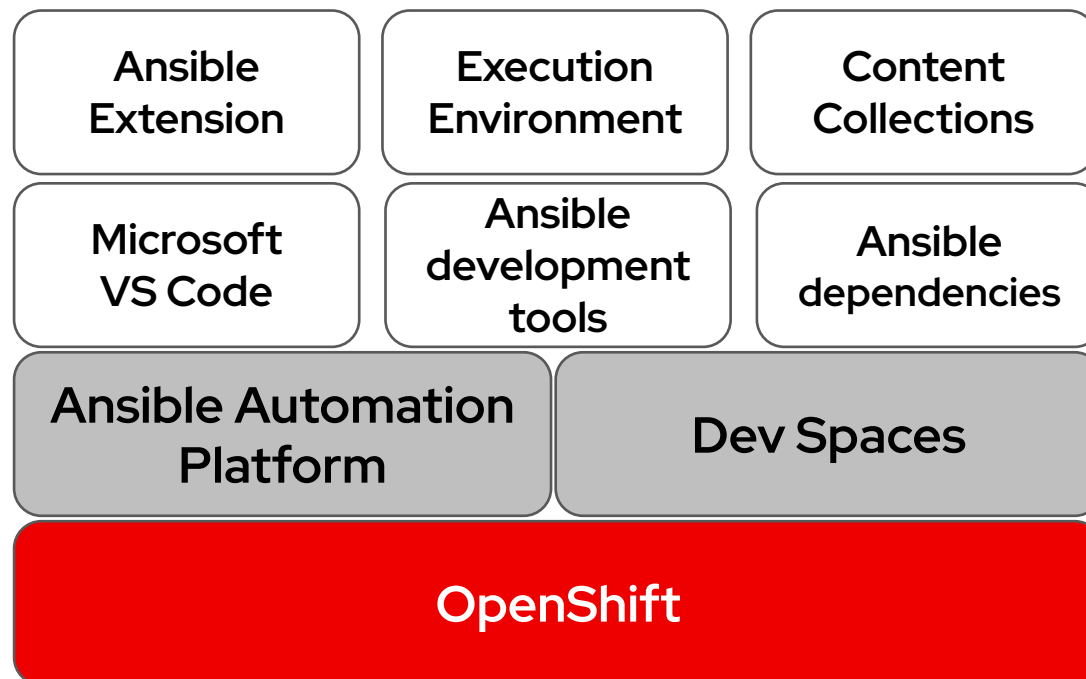


What are Ansible development workspaces?

Enterprise-ready on-demand Ansible content development environments

What is it?

- ▶ One-click provisioning of standardized, secure, environments based on Ansible best practices
- ▶ Everything automation developers need in a browser – VS Code, Ansible development tools, Ansible Lightspeed, documentation, etc.
- ▶ Supports Ansible content development on Windows and Mac OS workstations
- ▶ Enables IT organizations centralized management and policy enforcement



Within Red Hat OpenShift, launch Dev Spaces

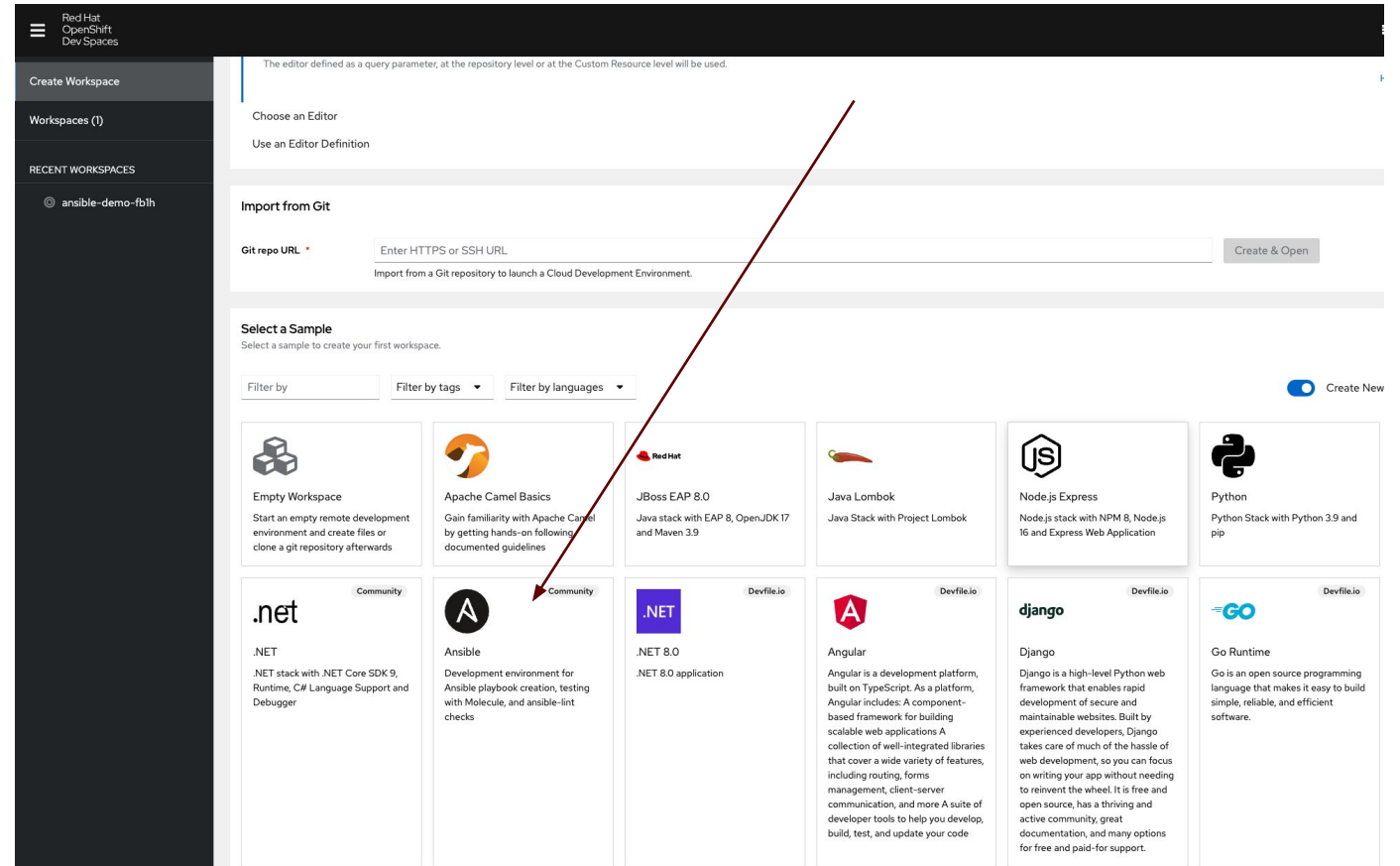
The screenshot displays the Red Hat OpenShift console interface. On the left is a sidebar with navigation links: Home, Favorites, Ecosystem, Helm, Workloads, Networking, Storage, Builds, Pipelines, Observe, Compute, User Management, and Administration. The main content area is titled 'Overview' and shows a 'Cluster' tab. A red arrow points from the 'Overview' title to a dropdown menu that is open, listing several options: Red Hat Applications, OpenShift Cluster Manager, Red Hat Hybrid Cloud Console, Red Hat OpenShift Dev Spaces (highlighted with a blue link icon), OpenShift GitOps, and Cluster Argo CD. Below the dropdown, the 'Overview' page contains sections for 'Getting started resources' (including 'Set up your cluster' and 'Build with guided documentation'), 'Details' (showing Cluster API address and ID), 'Status' (with a grid of health indicators for Cluster, Control Plane, Operators, Insights, Dynamic Plugins, and Storage), and 'Activity' (showing ongoing activities and recent events).

In Dev Spaces, launch Ansible workspace

launch provided creator image

Sample is provided

- ▶ Ansible creator image
- ▶ Contains ansible development tools
- ▶ Batteries included



Launching Workspaces

Automatic setup

- Creates all the necessary objects
- Provisions storage, network and compute
- Copies secrets and configurations

Workspaces > Starting workspace ansible-demo-e6nf

Starting workspace ansible-demo-e6nf

Starting

Progress

Logs

Events

✓ 1 Initializing

✓ 2 Checking for the limit of running workspaces

✓ 3 Creating a workspace

4 **Waiting for workspace to start**

✓ DevWorkspace is starting

✓ Resolved plugins and parents from DevWorkspace

✓ Storage ready

✓ Networking ready

✓ DevWorkspace serviceaccount ready

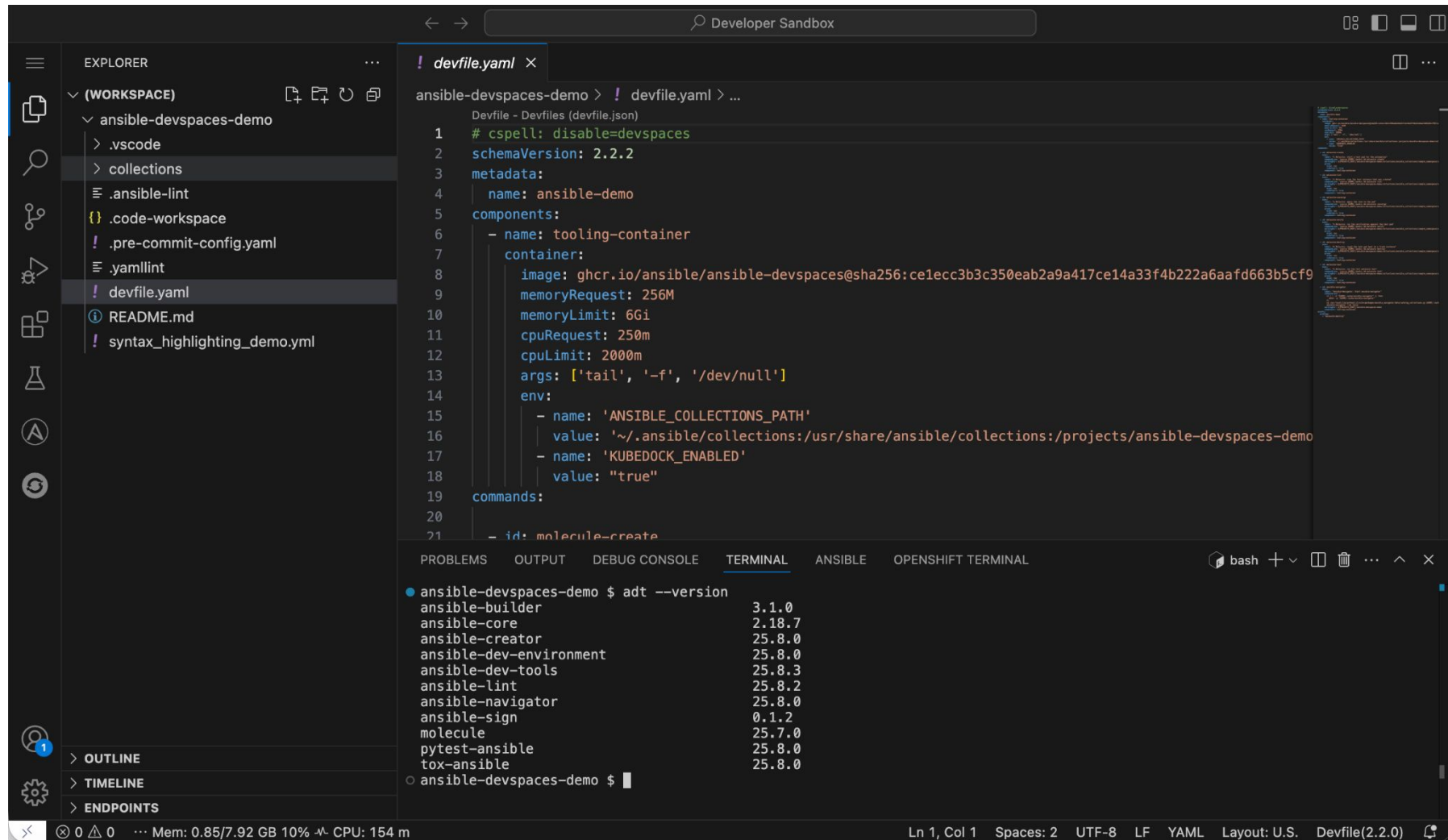
✓ DevWorkspace secrets ready

Waiting for workspace deployment

🔄 **Downloading IDE binaries... (it can take a few minutes)**

5 Open IDE

Familiar Interface



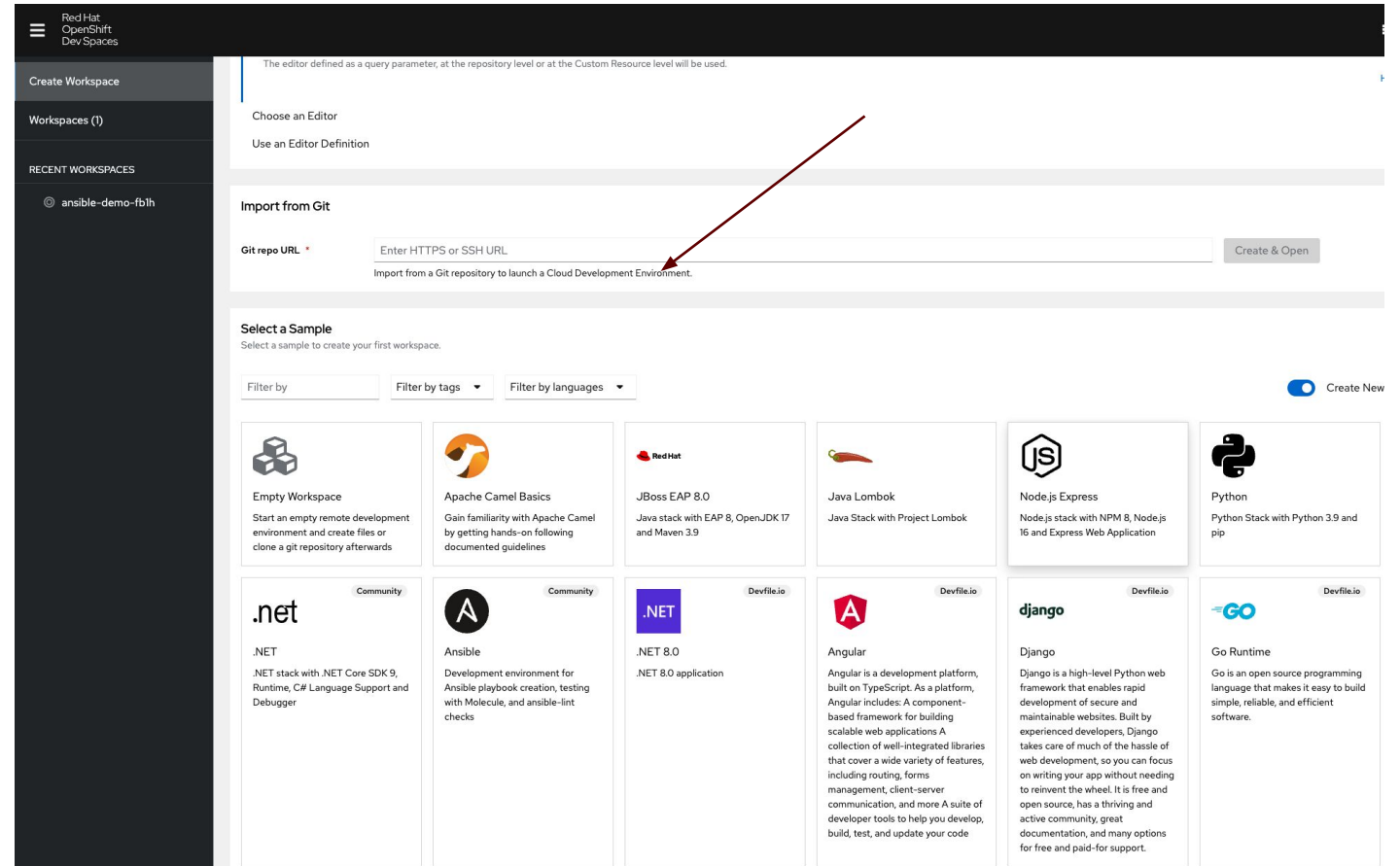
Dev Spaces for Power Users

Dev Spaces for Power Users

Advanced Capabilities

Customizable Quick Start Samples

- ▶ Samples are configurable
- ▶ Batteries included
- ▶ Custom tasks can be included
- ▶ Additional permissions and/or resources configured

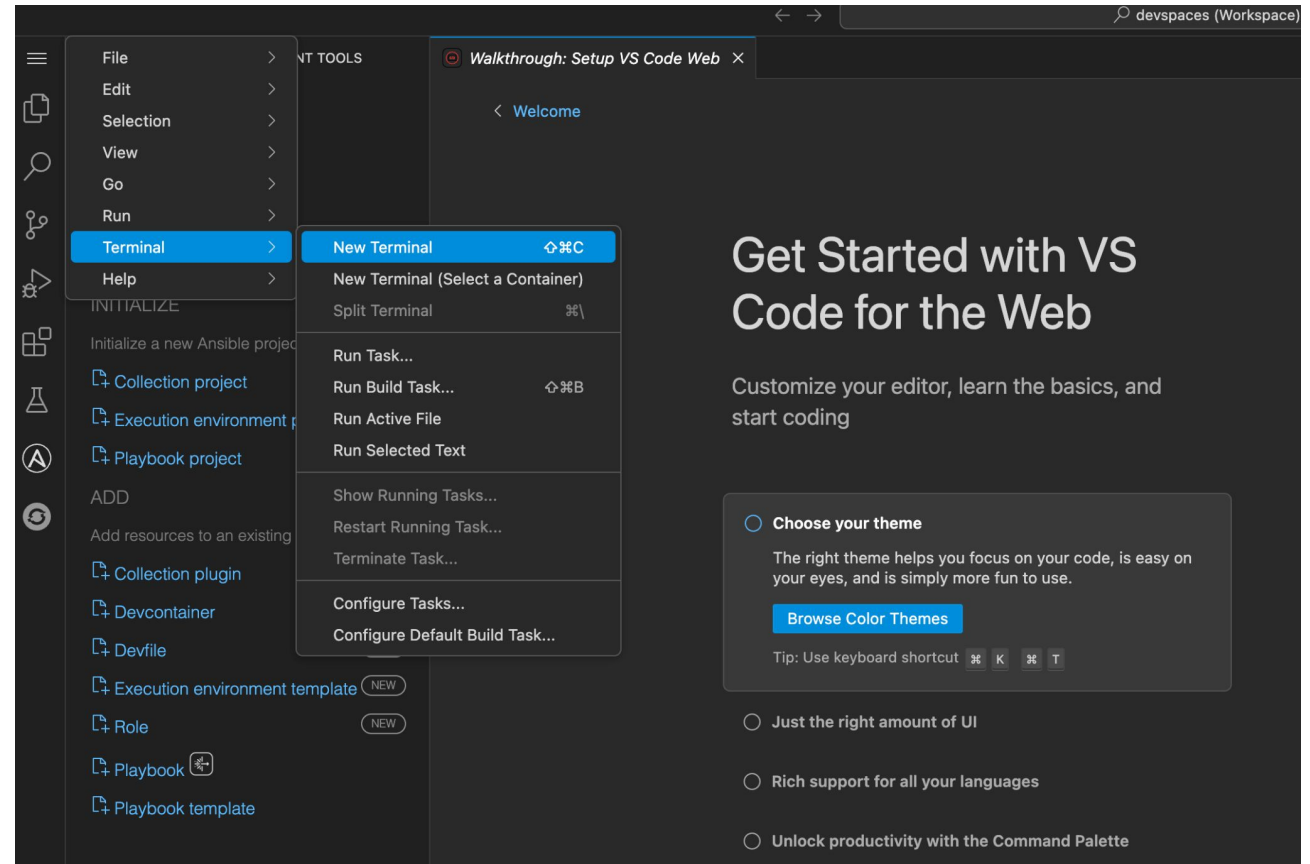


Dev Spaces for Power Users

Advanced Capabilities

Fully Functional Terminal

- ▶ No need to install things locally
"command not found: ansible"
- ▶ Workspace terminal inherits user's OCP permissions
"oc whoami"



Dev Spaces for Power Users

Advanced Capabilities

Integration with Source Control

- ▶ Automatically setup credentials for GitLab, GitHub, Bitbucket and Azure
- ▶ Leverages OAuth integration to reduce dependency on static secrets installed on laptops/servers

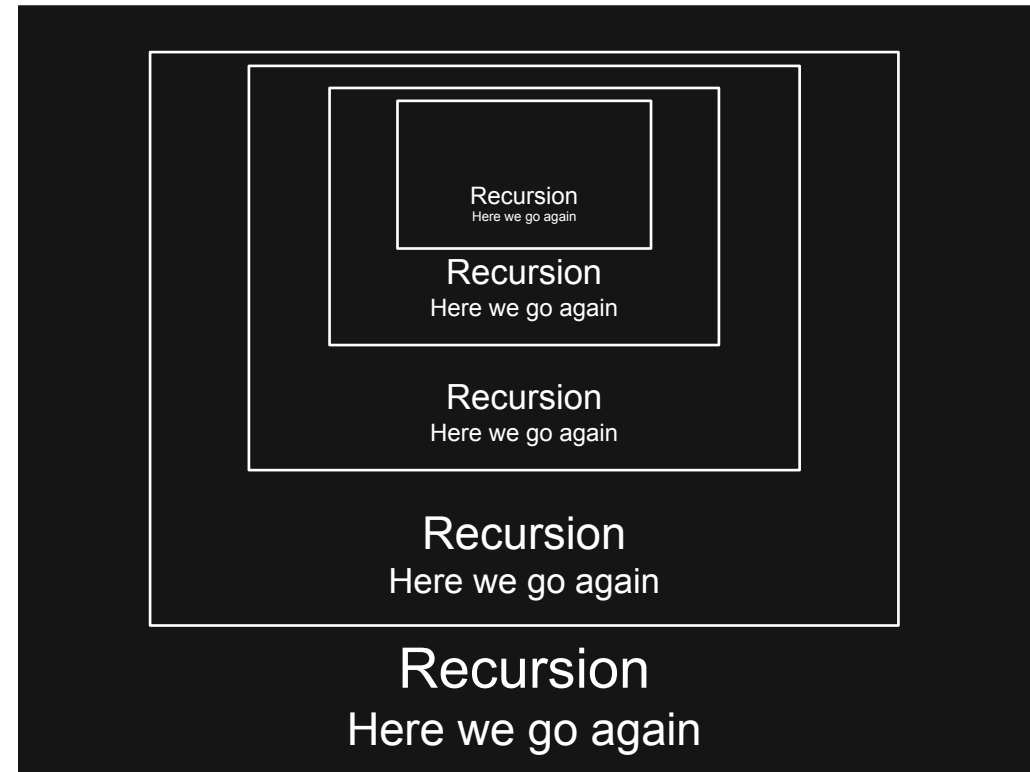


Dev Spaces for Power Users

Advanced Capabilities

Linux User Namespaces

- ▶ GA in OpenShift 4.20
- ▶ **Nested containers** now supported
- ▶ Ansible-navigator using EEs - no need for KUBEDOCK
- ▶ Ansible-builder, podman, buildah and more!



Dev Spaces for Power Users

Advanced Capabilities

Quick Launch in Dev Spaces

- Add to repositories to automatically launch workspaces

 README



Ansible Development on OpenShift Dev Spaces

This repository provides a development environment for Ansible playbook creation, testing with Molecule, and ansible-lint checks using OpenShift Dev Spaces.

Summary

This repository contains a `devfile.yaml` file, which defines the development environment for Ansible. The DevSpace created using this `devfile` provides the necessary tools and dependencies for Ansible playbook development, testing with Molecule, and linting with ansible-lint. This is designed to be used in environments where developers do not have easy access to linux systems from which to develop ansible automation content, but do have OpenShift.

Dev Spaces for Power Users

Preconfigured Workspaces

Apply Organization Settings

- ▶ Recommended extensions
- ▶ Required settings
- ▶ Pre-commit hooks

📖 README 📄 GPL-3.0 license



Ansible Development Workspace Sample

Welcome to the Ansible Development Workspace sample repository! This repository provides a ready-to-use, containerized development environment for Ansible content creation, testing, and debugging using Red Hat OpenShift Dev Spaces.

By importing this repository into Dev Spaces, you instantly get a pre-configured, browser-based IDE equipped with all the essential tools you need to build high-quality automation—without installing anything on your local machine.

What's Included

This environment is configured via the included `devfile.yaml` and `.code-workspace` files. These files setup your workspace to run an Ansible specific Dev Spaces image and configure the IDE with recommended extensions and settings specific to Ansible development.

Ansible Testing

Categories of Testing



Sanity Testing

Ensure that the automation meets industry standards & best practices

Unit Testing

Validate that components of the code are rapidly tested independently; Test for scenarios that wouldn't be caught in integration testing (ie. Y2K type scenarios)

Integration Testing

Validate that the code functions as expected against a "real world" replicated environment

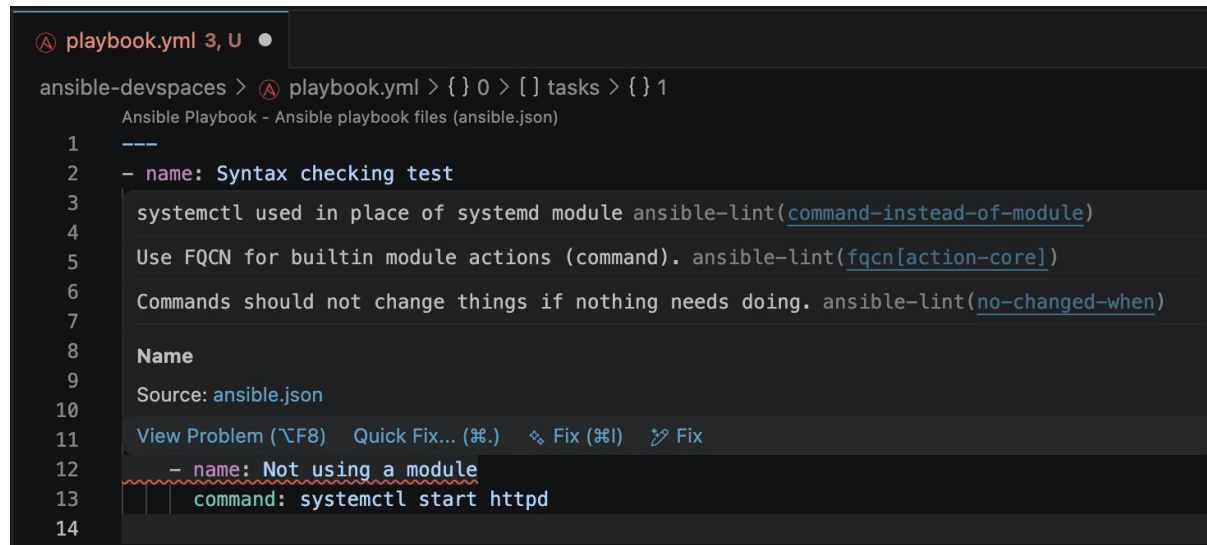


Continuous Integration Testing

Automation Code Validation and Testing

- ▶ Automation code requires testing
- ▶ Pipelines are as important for automation as applications
- ▶ Modern CI platforms provide the means to run sanity, unit and integration testing as part of a pipeline

Red Hat Ansible IDE Plugin



The screenshot shows the Red Hat Ansible IDE Plugin interface. The top bar indicates the current file is `playbook.yml` with 3 errors and 0 warnings. The breadcrumb navigation shows the path: `ansible-devspaces > playbook.yml > {} 0 > {} tasks > {} 1`. The main editor displays the content of `ansible.json`, which is an Ansible playbook. The playbook has a task named "Syntax checking test" that uses the `ansible-lint` module. The task's `command` is `systemctl start httpd`. The interface highlights the `command` property and shows a linting error: "Not using a module". The error message is: "systemctl used in place of systemd module ansible-lint(command-instead-of-module)". The interface also shows a list of built-in modules: `ansible.builtin.systemd`, `ansible.builtin.systemd_service`, `ansible.builtin.eos_system`, `ansible.builtin.icx_system`, `ansible.builtin.ios_system`, `ansible.builtin.net_system`, `ansible.builtin.cnos_system`, `ansible.builtin.nxos_system`, `ansible.builtin.vyos_system`, `ansible.builtin.iosxr_system`, `ansible.builtin.junos_system`, and `ansible.builtin.filesystem`.

```
1 ---
2 - name: Syntax checking test
3   ansible-lint(command-instead-of-module)
4   Use FQCN for builtin module actions (command). ansible-lint(fqcn[action-core])
5   Commands should not change things if nothing needs doing. ansible-lint(no-changed-when)
6
7   Name
8   Source: ansible.json
9
10  View Problem (⌘F8) Quick Fix... (⌘.) Fix (⌘I) Fix
11
12  - name: Not using a module
13    command: systemctl start httpd
14
```

- Syntax Highlighting
- Validation
- Ansible/YAML linting
- Smart auto-completion
- Hover documentation reference
- Jump to module code

Ansible Molecule



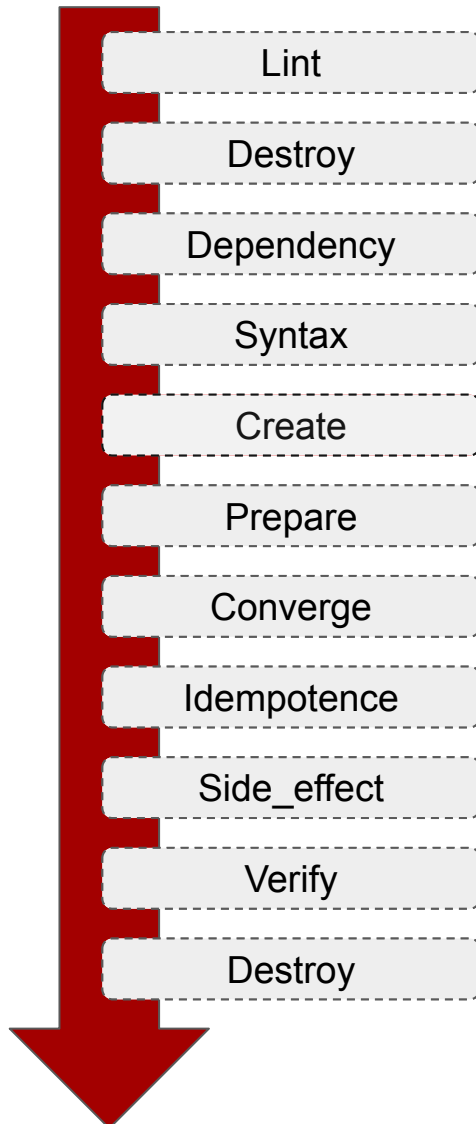
```
Molecule Scenario Schema (molecule.json)
1  ---
2  dependency:
3    name: galaxy
4  driver:
5    name: delegated
6    options:
7      managed: true
8      ansible_connection_options:
9        connection: local
10 platforms:
11   - name: molecule-rhel8-python-39
12     image: registry.redhat.io/rhel8/python-39
13     workingDir: '/tmp'
14   - name: molecule-rhel8-python-38
15     image: registry.redhat.io/rhel8/python-38
16     workingDir: '/tmp'
17 scenario:
18   name: default
19   test_sequence:
20     - lint
21     - syntax
22     - prepare
23     - create
24     - converge
25     - idempotence
26     - verify
```

- Simple to use Ansible testing tool
- Works for playbooks and roles
- Creates and destroys test systems
- Built in idempotency testing
- Sanity and integration testing
- Customizable verification



Molecule Scenarios

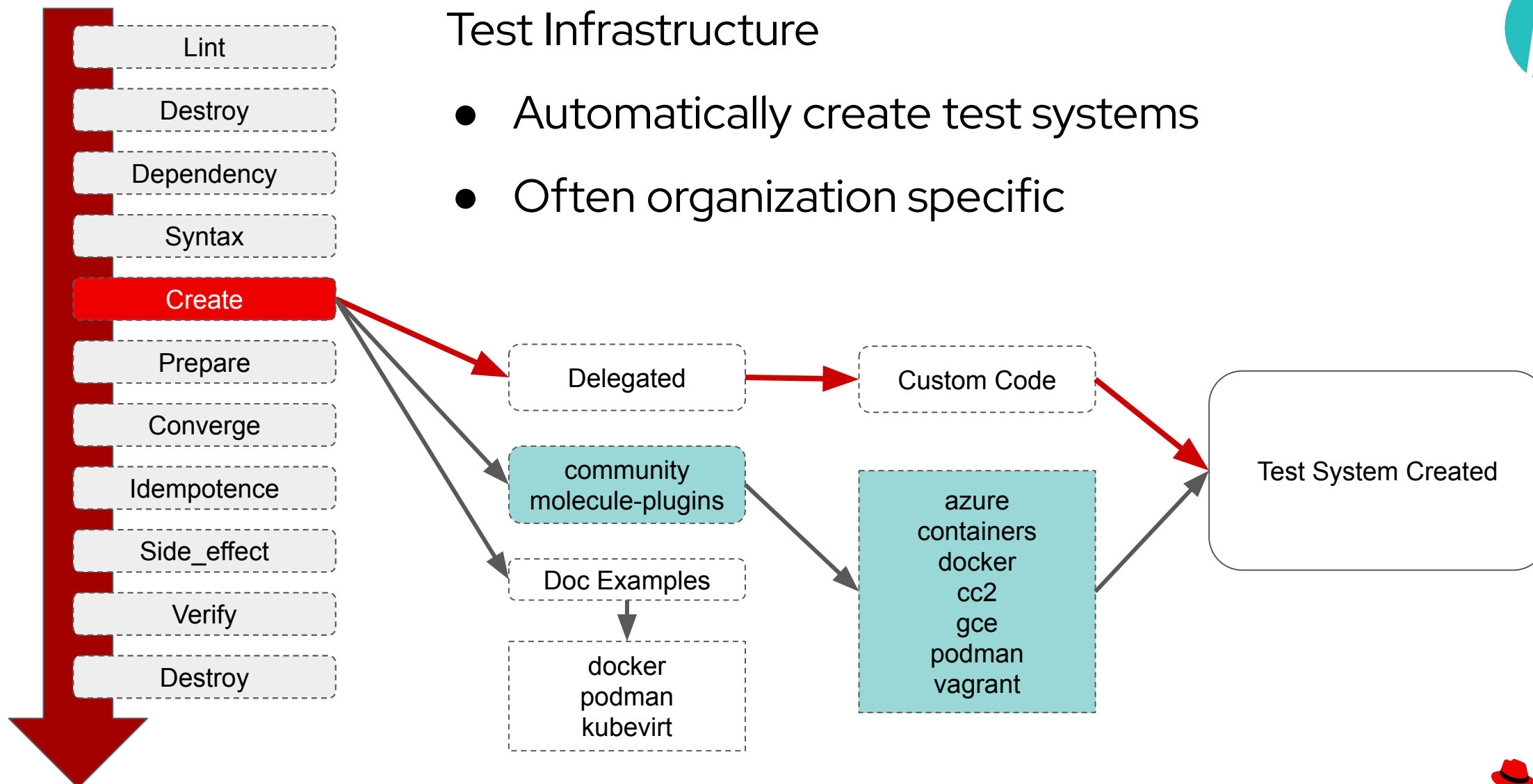
- Configurable steps
- Highly customizable
- Creates, tests and destroys systems





Test Infrastructure

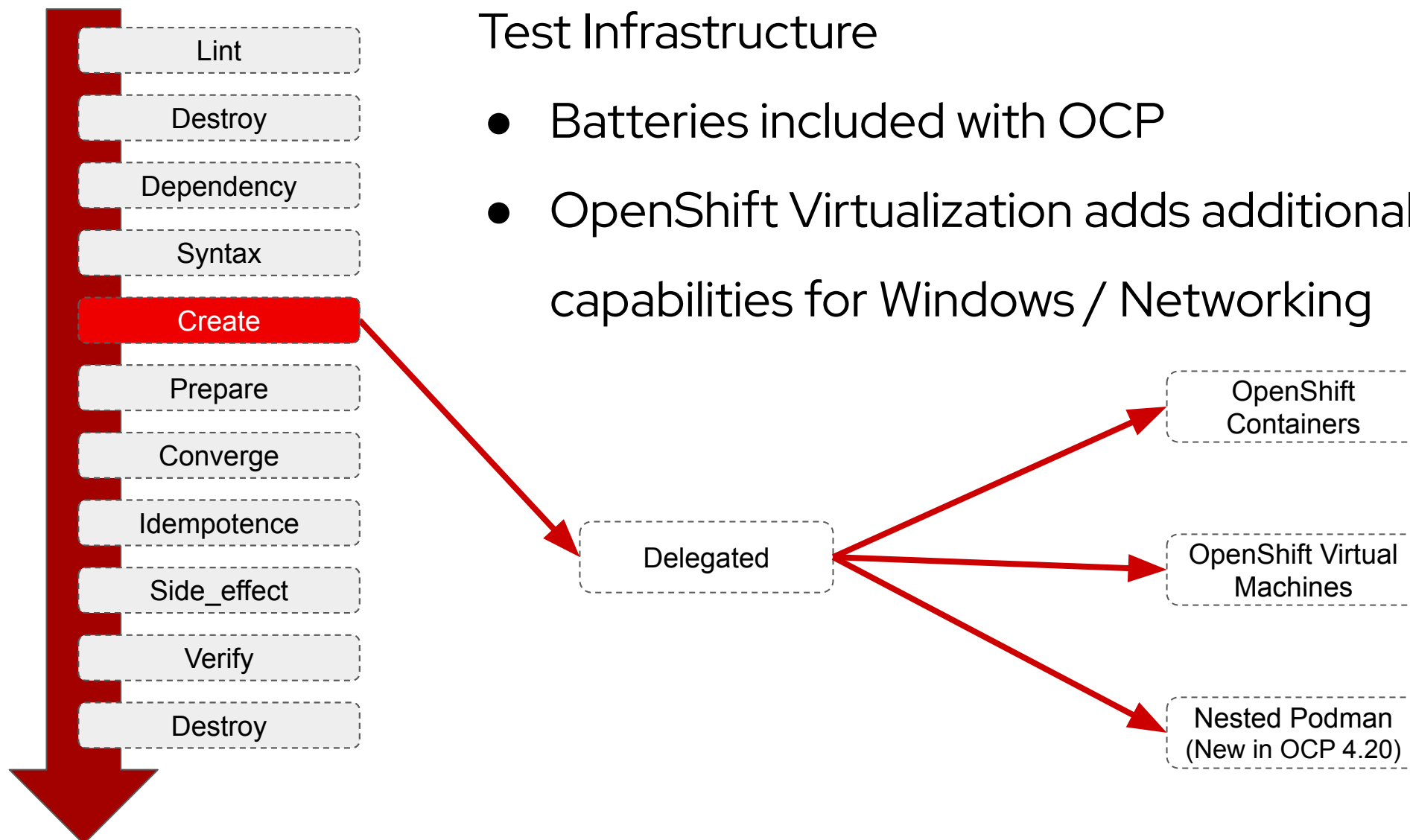
- Automatically create test systems
- Often organization specific





Test Infrastructure

- Batteries included with OCP
- OpenShift Virtualization adds additional capabilities for Windows / Networking



Other Important Tools

Changelog

7.5.0 (2023-04-17)

Features

- [503a4e5](#) [#548](#) allow identifierBase to be false ([#548](#)) (@lsvalina)

Bug Fixes

- [e219bb4](#) [#552](#) throw on bad version with correct error message (
- [fc2f3df](#) [#546](#) incorrect results from diff sometimes with prerelea
- [2781767](#) [#547](#) avoid re-instantiating SemVer during diff compare

Semantic Versioning / Changelog

Container Security
Scanner

Pre commit hooks

Podman / Docker
Desktop

CI/CD Pipeline Tool

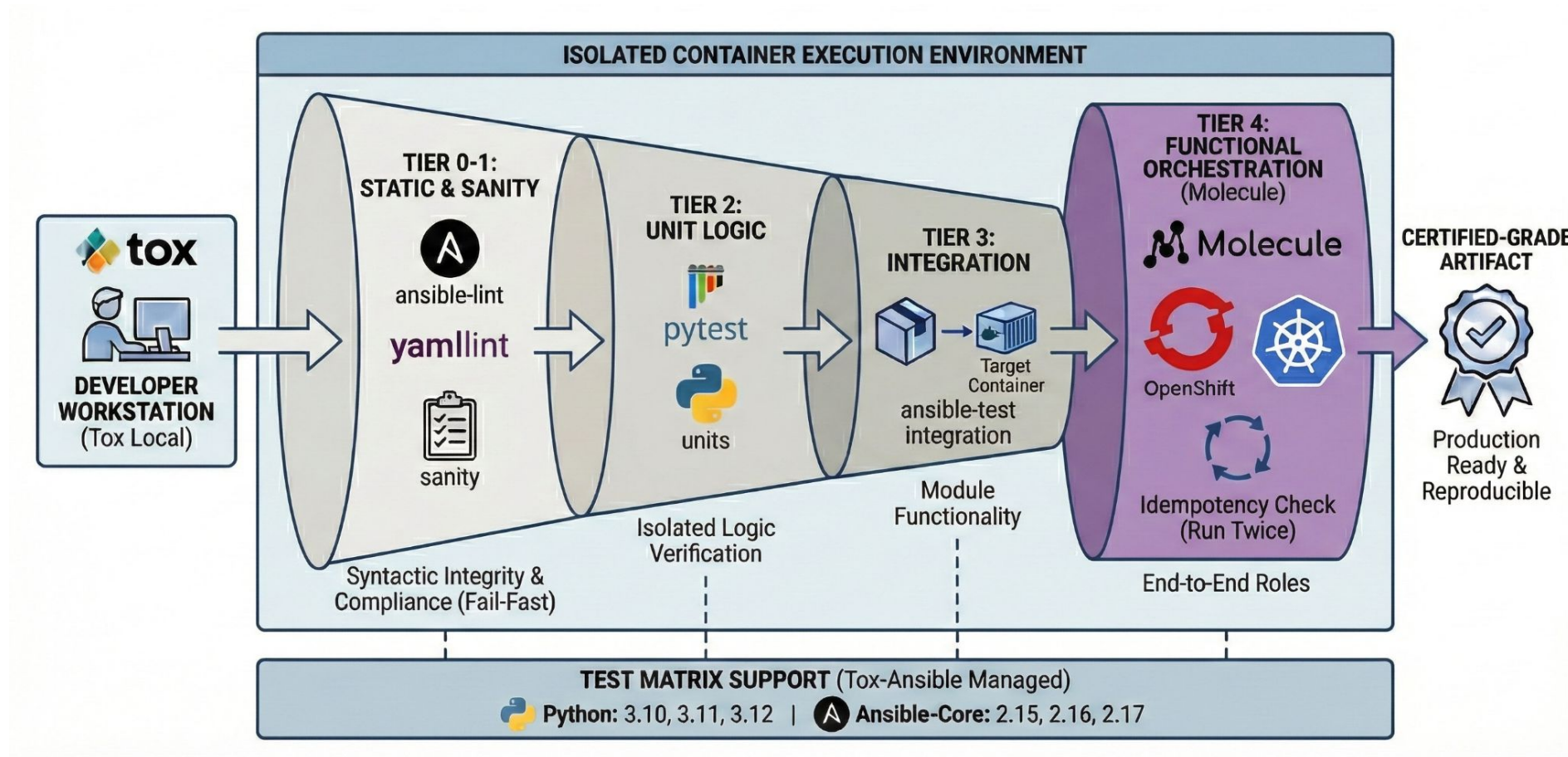


Secret Detection

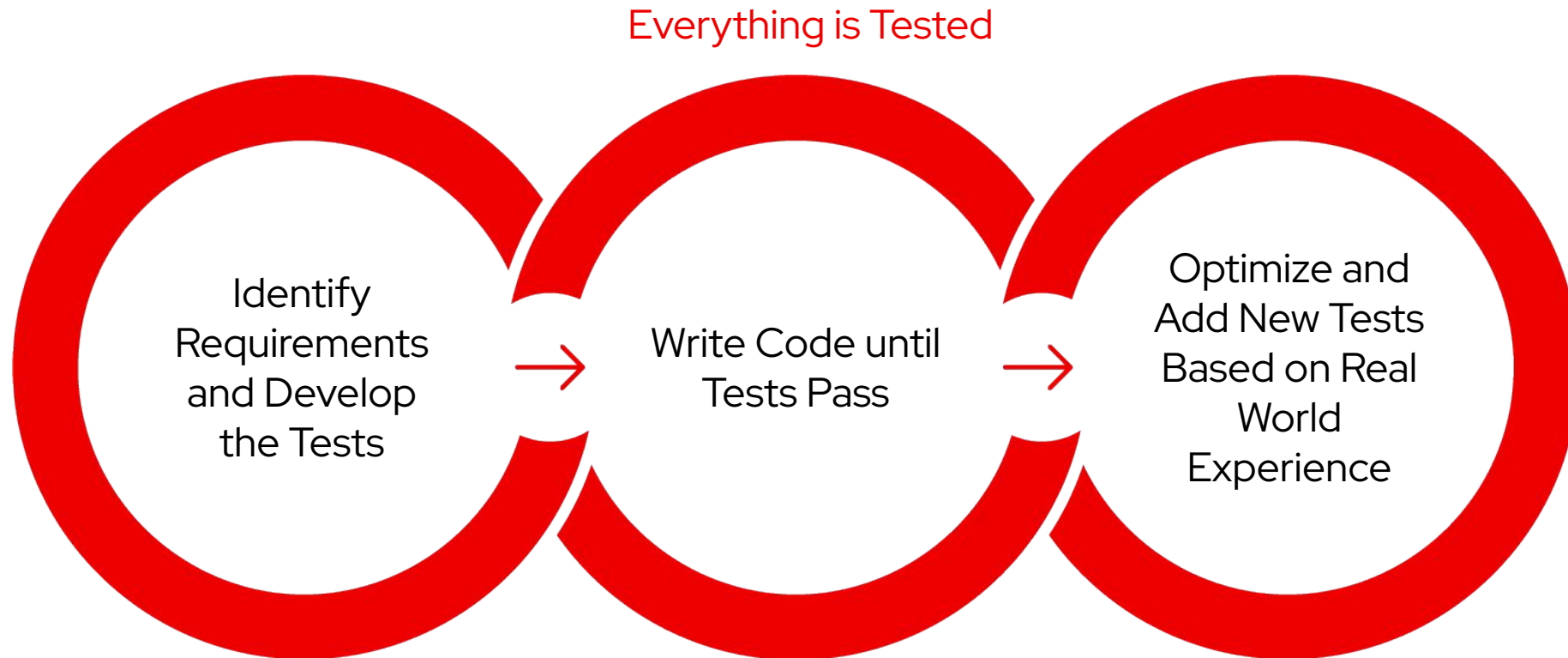
Secret Management
Tool

Test Infrastructure

Testing Pipelines



Test Driven Development



Getting started

Start Testing Today

Start exploring in the Developer Sandbox for free

Try Red Hat's products and technologies without setup or configuration.

[Start your sandbox for free](#)



Community

Ansible

Development environment for Ansible playbook creation, testing with Molecule, and ansible-lint checks



OpenShift

- ✓ Comprehensive cloud-native application platform
- ✓ Consistently develop and deploy applications at scale
- ✓ Streamline application development with CI/CD tools
- ✓ Manage containers, VMs, and serverless workloads across the hybrid cloud

[Try it](#)



OpenShift AI

- ✓ Scalable AI and ML platform
- ✓ Optimized for AI workloads
- ✓ Train, serve and monitor models
- ✓ Supports predictive and generative AI
- ✓ Scales across the hybrid cloud

[Try it](#)



Dev Spaces

- ✓ Cloud Development Environment
- ✓ Developer workspaces defined as code
- ✓ Kubernetes development made easy
- ✓ Near instant onboarding
- ✓ VS Code and JetBrains IDEs

[Try it](#)



Ansible Automation Platform

- ✓ Scalable, centralized automation solution
- ✓ Available on-prem, cloud, and hybrid
- ✓ Manage and monitor workflows, content, and execution
- ✓ Enforce policies and consistent configurations
- ⌚ 30-minute environment provisioning

[Provision](#)

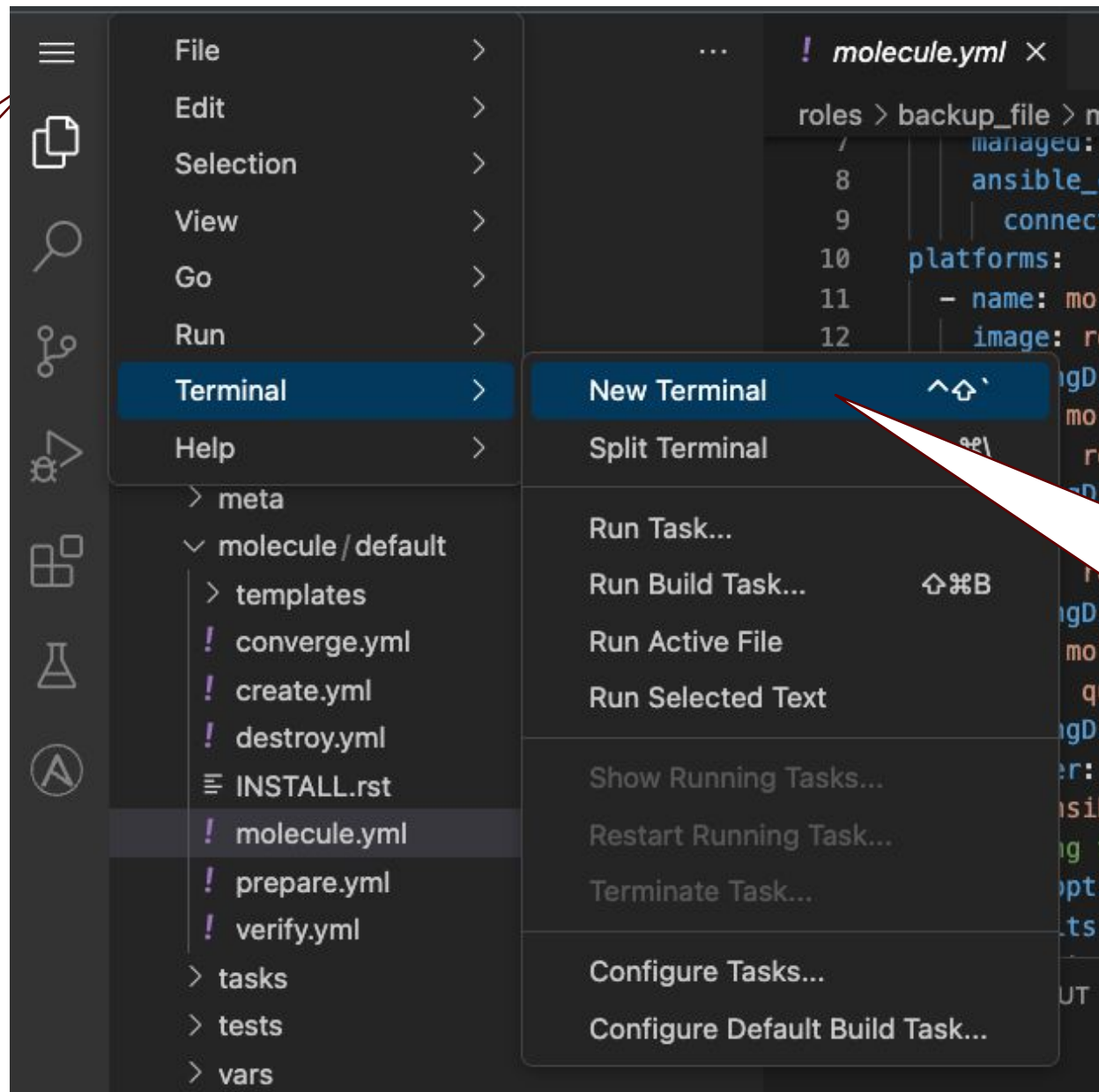


OpenShift Virtualization

- ✓ Migrate traditional VM workloads to OpenShift
- ✓ Unified platform for VMs, containers, and serverless workloads
- ✓ Supports modernizing application development
- ✓ Comprehensive development and operations tools

[Try it](#)

Click hamburger menu and select 'Terminal'



Click 'New Terminal' to get a fully configured linux terminal setup for Ansible development, regardless of the system you're on now

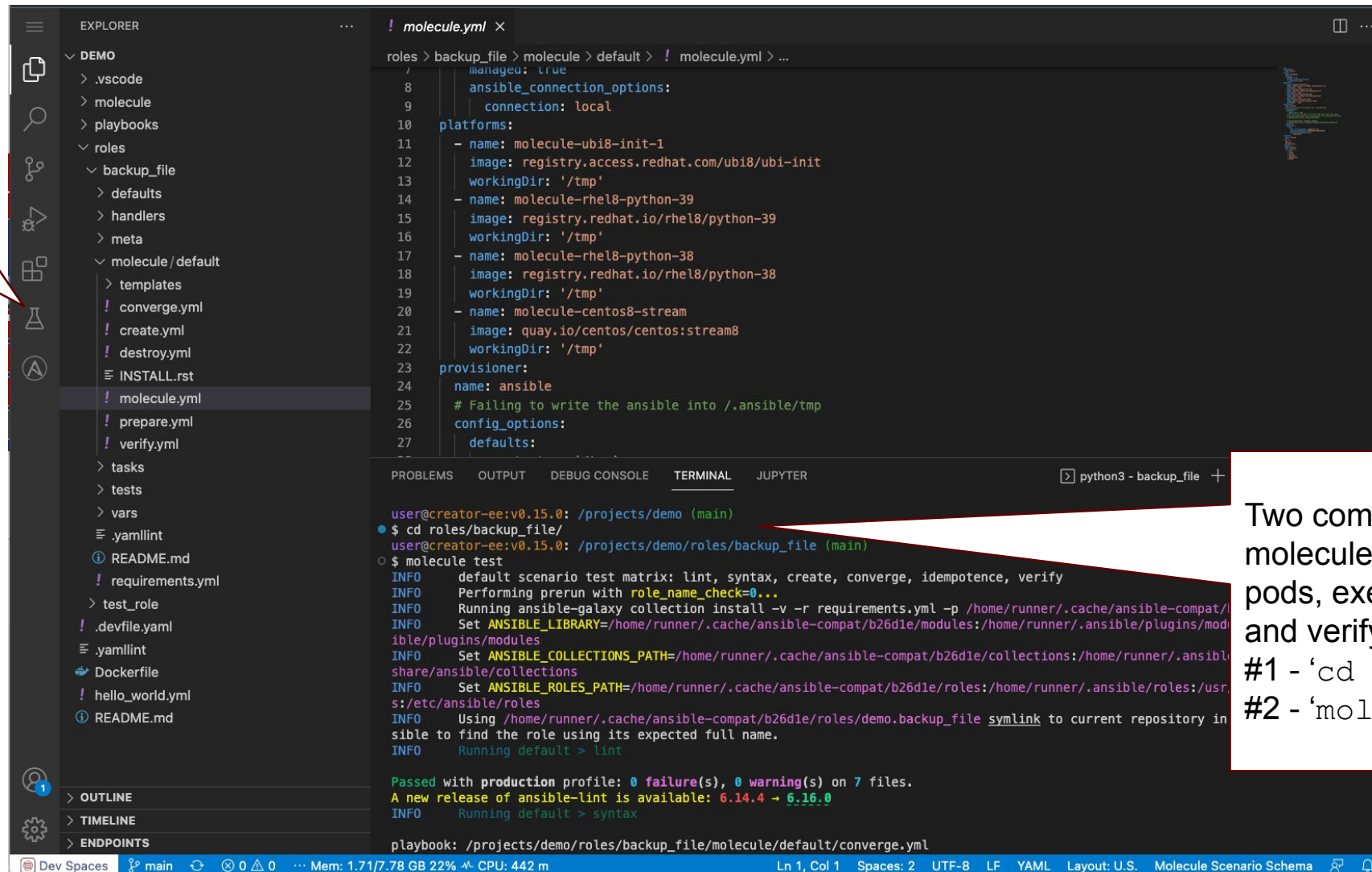


Developer Workspace

< - [Click here to see for yourself](#)



Full VScode experience including pre configured plugins and highlighting



Two commands to start running molecule tests that spin up OCP pods, execute the Ansible code and verify success.

#1 - 'cd roles/backup_file'
#2 - 'molecule test'